Distributes Systems — Module 2

A.Y. 2025/2026

Prof. Giovanni Ciatto, Tutor: Matteo Magnini

Compiled on: 2026-02-09 — printable version

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA
DEPARTMENT
OF COMPUTER SCIENCE
AND ENGINEERING

Table of Contents

Teachers

Prof. Giovanni Ciatto

- email: giovanni.ciatto@unibo.it
- homepage: https://www.unibo.it/sitoweb/giovanni.ciatto/en
- office hours: by appointment (send me an email)

Tutor Matteo Magnini

- email: matteo.magnini@unibo.it
- homepage: https://www.unibo.it/sitoweb/matteo.magnini/en
- office hours: by appointment (send me an email)

Prioritize the General forum

- All technical question
- Any other non-personal question

When using the email

- Include *all* teachers in CC, **always**, there including prof. Omicini
- Clarify the *academic year* and the *name of the course* in the subject

Pages of the course

- Institutional Page of the Course
- Virtuale Page of the Course
  - please enroll if you didn't already
- APICe Page of the Course
- These slides
- Examples and Exercises Repository
- Python Programming 101

Organization of Module 2

- Lectures in lab with immediate *hands-on*
- Exercises and examples in Python
  - submission of *exercises* via GitHub may grant you *extra points*
- Final project work in any programming language of your choice
  - report must be in English and follow this LaTeX template

Time—table

- **Friday 9:00–11:00** (2h) — Lab 3.1
  - cf. official timetable for any change

Changes will be published on the News forum

Workflow for the project work (pt. 1)

Detailed rules here: https://apice.unibo.it/xwiki/bin/view/Course/Ds2526/Projects/Rules

1. **Propose** your project idea on the Projects forum
   - 1-3 students per *group*
   - open a discussion thread named `[Surname1, Surname2, ...] Project Proposal: <your project name>`
   - please indicate:
     1. names and **email-addresses** of the members of the group;
     2. the project *vision*, i.e. what you want to realise, eliciting the *functionalities* of the envisioned system;
     3. a *motivation* of why/how the project is letting you *explore* the **topics of the course** (e.g. consistency, fault-tolerance, etc);
     4. *which technologies* (programming langues, libraries, technologies) you intend to use and *why*;
     5. which *deliverable(s)* you intend to produce (application, library, presentation, etc), **aside from the final report**;
     6. some strict *temporal constraint*, if any (e.g., someone in the group is going to graduate soon)
2. Any *subsequent communication* on the project should be done on the **same thread**
   - use the thread as a *backlog* of any official communication concerning the administration of your project work
   - if you want to talk *privately* about your project work, *mention the URL* of your project's thread in your email
3. **Wait** for the teachers to *approve* your proposal
   - most likely, we'll try to estimate the effort and ask for changes if too much or too little
   - we'll also check for overlaps with other groups

Workflow for the project work (pt. 2)

4. Create a **GitHub repository** where to develop your project and post the URL onto your project's thread
    - make all the *members* of the group collaborators on the repository
    - give *full admin* rights to the teachers:
        - Usernames: `gciatto`, `matteomagnini`
5. **Work** on your project:
    - *commit* and *push* your changes often
        - only the `master` or `main` branch will be considered for the evaluation
    - in parallel, or later, write a *report* about your report following this LaTeX template
6. Once done, post a *message* on your *project's thread* to signal the end of the work
    - we'll *clone* your repository and evaluate the code
7. **Edits** may be requested by the teachers, to either the code or the report (or both)
8. Once the teachers are satisfied, you'll be requested to **present** your work to prof. Omicini
    - after setting up a meeting with him, in private
    - 12-15 minutes *presentation* + 5-10 minutes *Q&A*

Goals

- Teach you how to **design** and **develop** Distributed Systems, in practice…
- … via a *running example* (namely, distributed Pong)
- Give you a *reference* for developing your final **project work**

Topics

- Distributed **architectures** and **protocols**
- Communication **mechanisms** (TCP / UDP *sockets*)
- *Presentation* (**serialization** and **deseralization**)
- **Issues** of distributed systems programming
- [Hopefully] **Testing** distributed systems
- [Hopefully] Overview on distributed programming **frameworks** / **libraries**

Prerequisites

- Basic knowledge of Python (see Python Programming 101 if missing)
- Basic knowledge of Git (see these slides if missing)
- Basic knowledge of Linux shell (see these slides if missing)
- Basic understanding of computer networks, ISO/OSI model, TCP/IP stack

Software Requirements

Required

- A working internet connection
- A working Python 3.10+ installation

Recommended

- PyCharm OR *Visual Studio Code*
- A decent Unix terminal
  - prefer *Git Bash* on Windows

Lecture is Over

Compiled on: 2026-02-09 — printable version

back to ToC